

Stealing sensitive browser data with the W3C Ambient Light Sensor API

Stealing sensitive browser data with the W3C Ambient Light Sensor API - Lukasz Olejnik, Security and Privacy Tech Inquiries.

- Published: 2017-04-19
- Original: <<https://blog.lukaszolejnik.com/stealing-sensitive-browser-data-with-the-w3c-ambient-light-sensor-api/>>
- Preserved from: <https://blog.lukaszolejnik.com/stealing-sensitive-browser-data-with-the-w3c-ambient-light-sensor-api/> (stored) on 2026-10-03
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[**

Back to main

](<https://blog.lukaszolejnik.com>) [Lukasz Olejnik](#)

In this post we describe and demonstrate a neat trick to **exfiltrate sensitive information from your browser** using a surprising tool: **your smartphone or laptop's ambient light sensor**.

In short:

- We provide background about the light sensor API and current discussions to expose it more broadly to websites.
- We show how the color of the screen of the user's device can affect light sensor readings, and explain why this has dangerous security and privacy consequences. We focus on **extracting browsing history** and **stealing data from cross-origin frames**, which makes it possible for attackers to discover the contents of sensitive documents and images (e.g. QR codes for account recovery).
- We discuss countermeasures that can be taken by browser vendors and specification maintainers to mitigate the risk.

The attack works in current versions of Firefox and Chrome (behind an experimental flag), on Android and desktop devices with light sensors, e.g. the MacBook Pro.

This post is a collaboration between myself, [@LukOlejn](#), and [@arturjanc](#) who built some cool [proofs of concept](#).

Background: Light sensors in smartphones

Today, all smartphones and a number of modern laptops are equipped with an ambient light sensor. This sensor is placed in the top region of a device, often close to the front camera. Smartphones have long taken advantage of ambient light readouts to adjust the display brightness to save battery, or for proximity detection. The information about light conditions can also be used to design responsive smartphone applications or to configure the hardware itself. Somewhat unsurprisingly, light sensor readouts can be sensitive, as discussed in my [previous post](#).

Data returned by the ambient light sensor is quite precise. The measured light intensity is in the SI unit [lux](#), and the output data ranges between 0 (dark) to tens of thousands of lux. The frequency of readouts from the sensor is relatively high, allowing reads at 100-200 millisecond granularity.

To better compete with native apps, websites might soon be able to access ambient light readings. There is currently an ongoing discussion within a W3C Device and Sensors Working Group whether to allow websites access the light sensor without requiring the user's permission. Most recent versions of both Chrome and Firefox have implementations of the API.

Sensor privacy

This research was spurred by a recent [W3C discussion](#) about the [generic sensor API](#), where one of the leading proposals posits that access to certain sensor data should not require asking the user for permission. As a response, we decided to investigate **the malicious potential behind ambient light sensors** (ALS); as a side note, I previously analyzed [security and privacy aspects of ALS](#), which may introduce new channels for tracking, profiling and data leaks: one example discussed “*detecting that two people are in the same room at the same time*”, another analyzed the possibility to [discover banking PINs](#).

In this post, we'll put such concerns aside and focus on how sensor readings can help an active attacker exfiltrate private data from the user's browser.

Now, let's focus on the actual attacks.

Exfiltrating information using the light sensor

How exactly can ambient light readings allow extracting private data? Our attack is based on two observations:

- The color of the user's screen can carry useful information which websites are prevented from directly accessing for security reasons.
- Light sensor readings allow an attacker to distinguish between different screen colors.

We cover the second point in more detail later, but in a nutshell, light emanating from the screen can affect light sensor readings in a measurable way, allowing a website to

determine the color of the user's screen (or, in the simplest case, distinguish a black screen from a white one).

The first point might be the more surprising one; after all websites control what they display on the user's screen, so why would this be interesting data? There are two main cases where the website cannot directly obtain the color of part of the user's screen:

- **Colors of visited links.** For privacy reasons, browsers lie to developers about the colors of links displayed on a page; otherwise a malicious developer could apply `:visited` styles and detect which websites are present in the user's history.
- **Cross-origin resources.** The [same-origin policy](#) prevents *evil.com* from directly accessing resources from *victim.com*. However, while sites cannot examine frames or images from other origins, they can display them on the screen and modify how they appear to users, including arbitrarily scaling them and changing their color.

Our attack relies on displaying such resources on the screen and detecting their color, leaking one bit of information at a time. Here's how this works in practice (using our early demonstration with increased verbosity):

Detecting visited links

Since a website can apply different styles to visited and unvisited links, but cannot detect how the links are displayed to the user, we use the sensor to identify its true color:

- *Set link styles:* visited (white), unvisited (black).
- *Calibrate:* display a white background, then follow with a black background to identify the light levels in the user's environment; this is also possible, but more difficult, if sensor readings fluctuate significantly.
- *Iterate through a list of links,* one by one, displaying each of them in turn as a large rectangle which fills the entire screen. Visited links will show up as white, unvisited as black.
- Log the light levels in response to displaying each link, identifying its color. Since we calibrated the screen in step #2, we know which color each reading represents.

At the end the attacker obtains a list of links for which the screen was white and knows that the user had previously visited the given pages.

Although in our proof of concept demonstrations we rely on the assumption that the light conditions do not change during the exfiltration phase, extending the demos to handle these situations shouldn't be a problem. In the first demonstration, we want to highlight the fact that the light readout are subject to change. However, in practice it's not difficult to overcome this obstacle.

Stealing cross-origin resources

Potentially more troubling is the fact that attackers can extract pixel-perfect representations of cross-origin images and frames: essentially, discover how a given site or image looks for the attacked user (in our demo we focus on images because they are easier to exfiltrate). In extreme cases, for example on sites which use account recovery QR

codes for emergency access to an account (<https://victim.com/account-code.png>), this could allow the attacker to hijack the victim's account.

The attack is similar to the technique from the [Pixel Perfect Timing](#) paper and works as follows:

- Embed an image from the attacked domain; generally this will be a resource which varies for different authenticated users such as the logged-in user's avatar or a security code.
- Use an SVG filter to create a monochrome representation of the image (filters in SVG work on cross-origin resources).
- Scale the image so that each pixel is expanded to fill up the entire screen.
- Iterate over all pixels in the image, displaying each of them on the user's screen, and log the light sensor reading for this pixel.
- Compose the result image from individual pixel readings.



This allows the **exfiltration of all image resources and data from any document** which allows cross-domain iframing (those without the X-Frame-Options header or the frame-ancestors Content Security Policy directive).

Attack details and other considerations

Now that we've demonstrated the attacks we discuss some of the considerations for practical exploitation of this technique.

Speed of detection

Since we extract one bit of information at a time, the main limiting factor in an exploit is the speed of detection. In principle, browser sensors can deliver a 60 Hz readout rate. However, this does not mean that we can actually extract 60 bits per second - that's because the ultimate detection limit is tied to the rate at which a change in screen brightness can be detected by the sensor. In our experimentation we measured a screen brightness to readout latency of 200-300ms, and for a fully reliable exploit it's more realistic to assume one bit per 500ms.

At this rate, example detection times are as follows:

- Plain text string of 8 characters: 24 seconds (assuming [6 bits per character](#) for an alphanumeric string rendered in a known font)
- Plain text string of 16 characters: 48 seconds
- 20x20 QR code: 3 minutes 20 seconds
- Detecting 1000 popular URLs in the history: 8 minutes 20 seconds
- 64x64 pixel image: 34 minutes 8 seconds

It is unlikely that the user will put up with a website which seems to arbitrarily switch the screen color between black and white long enough to allow detection of anything more than very short text strings. However, in some cases where a user leaves their device unattended (e.g. puts the device on a shelf at night) an attacker will be able to extract much more data; to make this practical a malicious site can use the [screen.keepAwake](#) API to keep the display on indefinitely.

In the future, light sensors will likely be able to measure the intensity or particular color saturation (red, green, blue) which will allow faster attacks.

Accuracy of sensor readings

While the light sensor itself provides accurate readings, one difficulty for practical demonstrations of the technique are the changing lighting conditions throughout the duration of a test, and the possibility of the user moving the device around (especially a mobile phone).

We have found that the accuracy of results is affected by factors such screen brightness (a brighter screen helps the attack), proximity and angle of a reflecting surface above the sensor (a phone lying on a shelf reflecting light from a parallel surface produces good results; the lack of a reflecting surface makes the attack significantly harder), and the amount of ambient light (darker environments are less noisy and make detection easier).

Browser support

The attack works in two modern browsers: Firefox and Chrome currently ship the old API known as ambient light events, which we use in our demo - it **works without permission in Firefox** but currently still requires enabling the `chrome://flags#enable-experimental-web-platform-features` flag in Chrome. This old API will soon be replaced with a new Ambient Light Sensors API, which is currently implemented in Chrome behind a flag (`chrome://flags#enable-generic-sensor`), which is functionally equivalent for our purposes.

We filed security/privacy bug reports for [Firefox](#) and [Chrome](#).

Countermeasures

The current proposal argues that the following protections are sufficient:

- Limit the frequency of sensor readings (to much less than 60Hz)
- Limit the precision of sensor output (quantize the result)

To our knowledge, these precautions are meant as a response to a vulnerability report based on research findings indicating that it's possible to recover audio signal with the use of a [gyroscope](#). Our understanding is that the Chrome proposal attempts to apply the solutions to “[gyroscope misuse](#)” to other sensors, which might not be sufficient in all instances. Attempts to generalize threat models built on specific and specialized risks may not be correct.

For the light sensor, **limiting the frequency will not thwart our attacks**; even frequency as low as 1Hz would allow the same type of attack, with only factor-of-two slowdown.

Limiting precision of output might be a better solution, as long as it makes it impossible for the color of the screen to influence the output level of the sensor. There might be conditions under which attacks are still possible (bright screen, reflective surface very close to the screen), but they would become more difficult to conduct in practice.

Perhaps the most obvious solution is to **require the user to grant permission to the website requesting access to the sensor**, as is already the case for other features such as geolocation. It would also be prudent to expand the security and privacy considerations section in the Ambient Light Sensor API specifications to document the risk of attacks such as this one.

In Firefox, try disabling sensors in `about:config` - set `device.sensors.enabled` to “*false*”. In Chrome most users should currently be safe from this attack unless they manually enabled one of the experimental flags mentioned earlier.

To protect documents from being iframed by untrusted sites, set the X-Frame-Options: Deny HTTP header. This will also prevent other attacks such as clickjacking.

It is currently not possible to mitigate attacks on images as they are not subject to X-Frame-Options restrictions.

Summary

The demonstrated attack shows how **seemingly innocuous output** from the ambient light sensor can allow malicious sites to violate the same-origin policy, steal cross-origin data and extract information about the user's browsing history.

There is a lesson here that designing specifications and systems from a [privacy engineering perspective](#) is a complex process: decisions about exposing sensitive APIs to the web without any protections should not be taken lightly. **One danger is that specification authors and browser vendors will base decisions on overly general principles and research results which don't apply to a particular new feature** (similarly to how protections on gyroscope readings might not be sufficient for light sensor data).

Perhaps the more exciting lesson is that technology standards, privacy engineering and privacy impact assessments are fun and let us uncover subtle issues that have real-world consequences. If you need help with such work on your specification or feature, feel free to reach out to [me](#).

Archived from <https://blog.lukaszolejnik.com/stealing-sensitive-browser-data-with-the-w3c-ambient-light-sensor-api/>. Rendered offline from the local Markdown copy.